Exercício 08 - Junções e Agregações

Exercício 01 - INNER JOIN e autorrelacionamento

1. Listar pedidos com nome do cliente e status

```sql
SELECT
    p.id_pedido,
    c.nome AS nome_cliente,
    p.status
FROM PEDIDO p
INNER JOIN CLIENTE c
    ON p.id_cliente = c.id_cliente;
```

2. Listar os produtos e subtotais de cada item

```sql
SELECT
    pr.nome AS produto,
    ip.quantidade,
    ip.preco_unitario,
    ip.quantidade * ip.preco_unitario AS subtotal
FROM ITEM_PEDIDO ip
INNER JOIN PRODUTO pr
    ON ip.id_produto = pr.id_produto;
```

3. Montar o caminho PEDIDO → ITEM_PEDIDO → PRODUTO

```sql
SELECT
    p.id_pedido,
    pr.nome AS produto,
    ip.quantidade,
    ip.preco_unitario,
    ip.quantidade * ip.preco_unitario AS subtotal
FROM PEDIDO p
INNER JOIN ITEM_PEDIDO ip
    ON p.id_pedido = ip.id_pedido
INNER JOIN PRODUTO pr
    ON ip.id_produto = pr.id_produto;
```

4. Listar empregados e respectivos supervisores

Considerando uma tabela EMPREGADO com as colunas id_empregado, nome e id_supervisor:

```sql
SELECT
    e.nome AS empregado,
    s.nome AS supervisor
FROM EMPREGADO e
INNER JOIN EMPREGADO s
    ON e.id_supervisor = s.id_empregado;
```

Explique por que cada condição ON preserva a semântica

Cada condição ON relaciona os registros de acordo com o significado do relacionamento entre as tabelas:

- PEDIDO e CLIENTE: p.id_cliente = c.id_cliente relaciona cada pedido ao cliente que o realizou.
- ITEM_PEDIDO e PRODUTO: ip.id_produto = pr.id_produto relaciona cada item ao produto correspondente.
- PEDIDO e ITEM_PEDIDO: p.id_pedido = ip.id_pedido relaciona cada item ao pedido ao qual pertence.
- EMPREGADO e SUPERVISOR: e.id_supervisor = s.id_empregado relaciona cada empregado ao seu respectivo supervisor.

Cada condição ON preserva a semântica porque utiliza as chaves que representam os relacionamentos reais entre os registros, garantindo que os dados sejam associados corretamente.

Exercício 02 - Junções externas e ausência de correspondência

1. Liste todos os clientes, inclusive os que não possuem pedidos

```sql
SELECT
    c.id_cliente,
    c.nome,
    p.id_pedido
FROM CLIENTE c
LEFT JOIN PEDIDO p
    ON c.id_cliente = p.id_cliente;
```

2. Localize produtos nunca vendidos

```sql
SELECT
    pr.id_produto,
    pr.nome
FROM PRODUTO pr
LEFT JOIN ITEM_PEDIDO ip
    ON pr.id_produto = ip.id_produto
WHERE ip.id_produto IS NULL;
```

3. Reescreva um RIGHT JOIN equivalente com LEFT JOIN

Consulta original:

```sql
SELECT
    c.nome,
    p.id_pedido
FROM PEDIDO p
RIGHT JOIN CLIENTE c
    ON p.id_cliente = c.id_cliente;
```

Consulta equivalente usando LEFT JOIN:

```sql
SELECT
    c.nome,
    p.id_pedido
FROM CLIENTE c
LEFT JOIN PEDIDO p
    ON c.id_cliente = p.id_cliente;
```

4. Simule uma junção externa completa em duas tabelas

Como o MySQL não oferece suporte direto ao FULL OUTER JOIN, podemos simular o comportamento usando LEFT JOIN, UNION ALL e um segundo LEFT JOIN

```sql
SELECT
    c.id_cliente,
    c.nome,
    p.id_pedido
FROM CLIENTE c
LEFT JOIN PEDIDO p
    ON c.id_cliente = p.id_cliente

UNION ALL

SELECT
    c.id_cliente,
    c.nome,
    p.id_pedido
FROM PEDIDO p
LEFT JOIN CLIENTE c
    ON p.id_cliente = c.id_cliente
WHERE c.id_cliente IS NULL;
```

5. Explique por que um filtro no WHERE pode eliminar linhas preservadas

```sql
SELECT
    c.nome,
    p.id_pedido
FROM CLIENTE c
LEFT JOIN PEDIDO p
    ON c.id_cliente = p.id_cliente
WHERE p.status = 'Concluído';
```

Embora o LEFT JOIN preserve todos os clientes inicialmente, o filtro WHERE p.status = 'Concluído' elimina as linhas em que p.status é NULL.

Isso acontece porque a condição não é verdadeira para essas linhas. Consequentemente, clientes sem pedidos deixam de aparecer no resultado.

Exercício 03 - Agregações GROUP BY e HAVING

1. Calcule quantidade, menor, maior e média de preços por categoria

```sql
SELECT
    categoria,
    COUNT(*) AS quantidade,
    MIN(preco) AS menor_preco,
    MAX(preco) AS maior_preco,
    AVG(preco) AS media_preco
FROM PRODUTO
GROUP BY categoria;
```

2. Conte pedidos por cliente, incluindo zero

```sql
SELECT
    c.id_cliente,
    c.nome,
    COUNT(p.id_pedido) AS quantidade_pedidos
FROM CLIENTE c
LEFT JOIN PEDIDO p
    ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome;
```

3. Mostre apenas clientes com pelo menos dois pedidos

```sql
SELECT
    c.id_cliente,
    c.nome,
    COUNT(p.id_pedido) AS quantidade_pedidos
FROM CLIENTE c
INNER JOIN PEDIDO p
    ON c.id_cliente = p.id_cliente
GROUP BY c.id_cliente, c.nome
HAVING COUNT(p.id_pedido) >= 2;
```

4. Calcule o valor total por pedido usando ITEM_PEDIDO

```sql
SELECT
    p.id_pedido,
    SUM(ip.quantidade * ip.preco_unitario) AS valor_total
FROM PEDIDO p
INNER JOIN ITEM_PEDIDO ip
    ON p.id_pedido = ip.id_pedido
GROUP BY p.id_pedido;
```

5. Justifique cada uso de WHERE, GROUP BY e HAVING

WHERE - Filtra linhas antes do agrupamento. Ex.: WHERE preco > 100;

GROUP BY - Agrupa linhas que possuem valores iguais nas colunas especificadas. Ex.: GROUP BY categoria;

HAVING - Filtra os grupos depois do agrupamento, inclusive usando agregações. Ex.: HAVING COUNT(*) >= 2;

Desafio Integrador - Entregue oito consultas multi tabelas e indicadores

1. Duas consultas com INNER JOIN e qualificação completa

Consulta 1 — Pedidos com clientes e status

Pergunta: quais pedidos foram registrados, quem são os clientes e qual é o status de cada pedido?

```sql
SELECT
    PEDIDO.id_pedido,
    CLIENTE.nome AS nome_cliente,
    PEDIDO.status
FROM PEDIDO
INNER JOIN CLIENTE
    ON PEDIDO.id_cliente = CLIENTE.id_cliente;
```

Expectativa: uma linha para cada pedido que possua um cliente correspondente, exibindo seu identificador, o nome do cliente e o status.

Interpretação: permite acompanhar os pedidos por cliente e verificar suas situações. A qualificação completa (PEDIDO.id_pedido, por exemplo) deixa explícito de qual tabela vem cada coluna e evita ambiguidades.

Consulta 2 — Produtos, itens e subtotais

Pergunta: quais produtos aparecem nos itens dos pedidos e qual é o subtotal de cada item?

```sql
SELECT
    ITEM_PEDIDO.id_pedido,
    PRODUTO.nome AS nome_produto,
    ITEM_PEDIDO.quantidade,
    ITEM_PEDIDO.preco_unitario,
    ITEM_PEDIDO.quantidade * ITEM_PEDIDO.preco_unitario
        AS subtotal
FROM ITEM_PEDIDO
INNER JOIN PRODUTO
    ON ITEM_PEDIDO.id_produto = PRODUTO.id_produto;
```

Expectativa: uma linha para cada item com produto correspondente, mostrando pedido, produto, quantidade, preço unitário e subtotal.

Interpretação: ajuda a identificar os produtos vendidos e os valores dos itens. A condição ON associa cada item ao produto correto por meio de seu identificador.

2. Duas consultas com junções externas

Consulta 3 — Todos os clientes, inclusive sem pedidos

Pergunta: quais clientes estão cadastrados e quantos pedidos possuem, mesmo que não tenham realizado nenhum?

```sql
SELECT
    CLIENTE.id_cliente,
    CLIENTE.nome,
    PEDIDO.id_pedido
FROM CLIENTE
LEFT JOIN PEDIDO
    ON CLIENTE.id_cliente = PEDIDO.id_cliente;
```

Expectativa: todos os clientes aparecem. Quando um cliente não possui pedidos, PEDIDO.id_pedido aparece como NULL.

Interpretação: permite identificar clientes ativos e clientes que ainda não realizaram pedidos. O LEFT JOIN preserva todas as linhas de CLIENTE, mesmo sem correspondência em PEDIDO.

Consulta 4 — Produtos nunca vendidos

Pergunta: quais produtos nunca foram incluídos em um item de pedido?

```sql
SELECT
    PRODUTO.id_produto,
    PRODUTO.nome
FROM PRODUTO
LEFT JOIN ITEM_PEDIDO
    ON PRODUTO.id_produto = ITEM_PEDIDO.id_produto
WHERE ITEM_PEDIDO.id_produto IS NULL;
```

Expectativa: somente produtos sem correspondência na tabela ITEM_PEDIDO.

Interpretação: ajuda a identificar produtos sem vendas registradas, o que pode apoiar análises de estoque e catálogo. O LEFT JOIN preserva todos os produtos inicialmente; o WHERE ... IS NULL seleciona aqueles que não encontraram correspondência.

3. Uma consulta com autorrelacionamento

Consulta 5 — Empregados e respectivos supervisores

Pergunta: quem supervisiona cada empregado?

```sql
SELECT
    E.nome AS nome_empregado,
    S.nome AS nome_supervisor
FROM EMPREGADO AS E
INNER JOIN EMPREGADO AS S
    ON E.id_supervisor = S.id_empregado;
```

Expectativa: uma linha para cada empregado que tenha um supervisor correspondente.

Interpretação: a tabela EMPREGADO é utilizada duas vezes, com os aliases E e S. A condição ON relaciona o identificador do supervisor registrado para o empregado ao identificador desse supervisor na própria tabela.

Como o INNER JOIN exige correspondência, empregados sem supervisor não aparecem.

4. Duas consultas agregadas com LEFT JOIN e contagem correta de zero

Consulta 6 — Quantidade de pedidos por cliente

Pergunta: quantos pedidos cada cliente possui, incluindo os clientes com zero pedidos?

```sql
SELECT
    CLIENTE.id_cliente,
    CLIENTE.nome,
    COUNT(PEDIDO.id_pedido) AS quantidade_pedidos
FROM CLIENTE
LEFT JOIN PEDIDO
    ON CLIENTE.id_cliente = PEDIDO.id_cliente
GROUP BY
    CLIENTE.id_cliente,
    CLIENTE.nome;
```

Expectativa: uma linha por cliente, com a quantidade de pedidos correspondente.

Interpretação: o LEFT JOIN mantém todos os clientes. O GROUP BY agrupa seus pedidos, e COUNT(PEDIDO.id_pedido) conta apenas os identificadores não nulos.

Consulta 7 — Quantidade de itens registrados por produto

Pergunta: quantos itens de pedido estão registrados para cada produto, incluindo aqueles que nunca apareceram em pedidos?

```sql
SELECT
    PRODUTO.id_produto,
    PRODUTO.nome,
    COUNT(ITEM_PEDIDO.id_produto) AS quantidade_itens
FROM PRODUTO
LEFT JOIN ITEM_PEDIDO
    ON PRODUTO.id_produto = ITEM_PEDIDO.id_produto
GROUP BY
    PRODUTO.id_produto,
    PRODUTO.nome;
```

Expectativa: uma linha por produto. Produtos sem itens de pedido terão contagem igual a zero.

Interpretação: permite comparar a presença dos produtos nos registros de pedidos. A contagem representa o número de registros de itens, e não necessariamente a soma de unidades vendidas. Para contar unidades, seria necessário utilizar SUM(ITEM_PEDIDO.quantidade).

5. Uma consulta que combine WHERE e HAVING

Consulta 8 — Categorias com pelo menos dois produtos acima de determinado preço

Pergunta: quais categorias possuem pelo menos dois produtos com preço superior a R$ 100?

```sql
SELECT
    PRODUTO.categoria,
    COUNT(*) AS quantidade_produtos,
    AVG(PRODUTO.preco) AS preco_medio
FROM PRODUTO
WHERE PRODUTO.preco > 100
GROUP BY PRODUTO.categoria
HAVING COUNT(*) >= 2;
```

Expectativa: somente categorias com pelo menos dois produtos cujo preço seja superior a R$ 100, exibindo a quantidade desses produtos e a média de seus preços.

Interpretação:

- WHERE filtra os produtos com preço acima de R$ 100 antes da agregação.
- GROUP BY agrupa os produtos restantes por categoria.
- HAVING mantém somente as categorias com pelo menos dois produtos após o filtro.

A média calculada considera apenas os produtos que passaram pelo WHERE, e não todos os produtos da categoria.